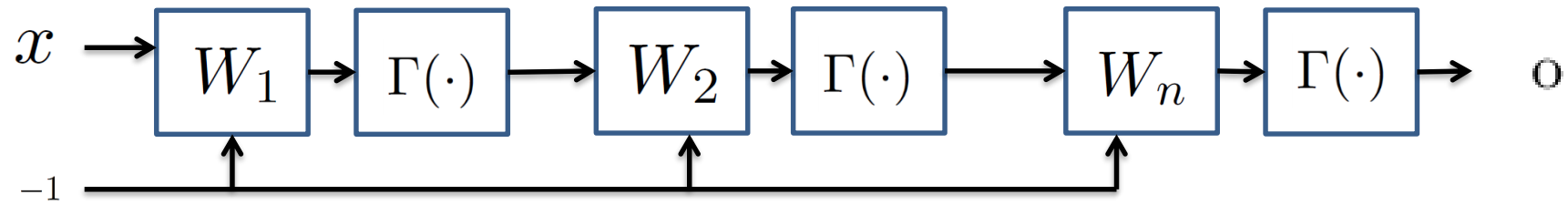


LEARNING

- Basics/Formulation
- Basic Learning Types
- Supervised Learning Rules
- Unsupervised Learning-Clustering
- General Form of Learning
- Hebbian Learning
- Winner Take All Learning
- Delta Learning Rule

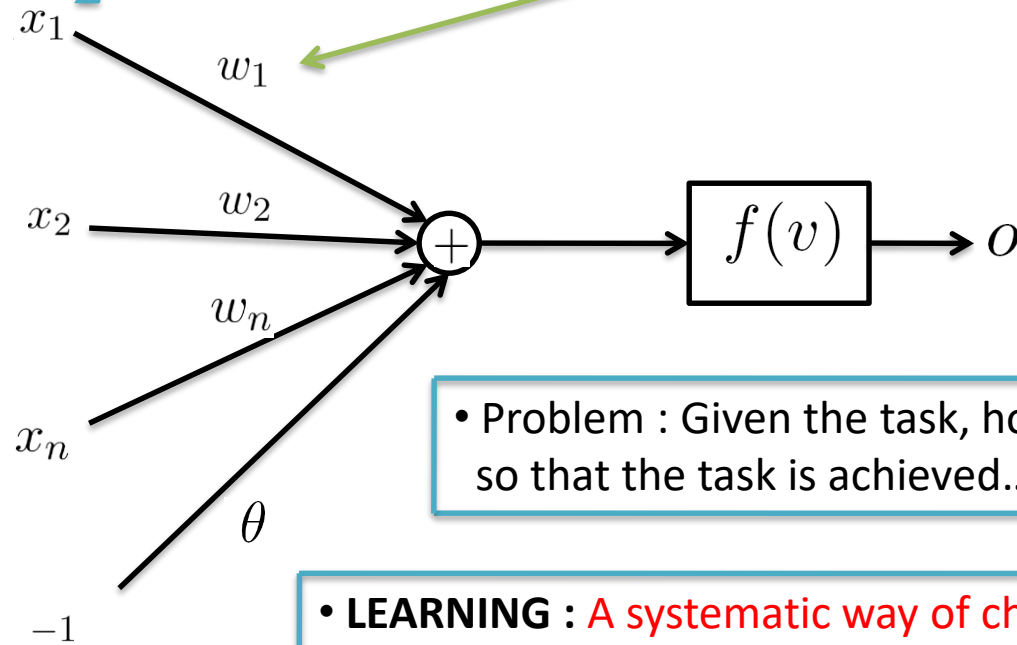
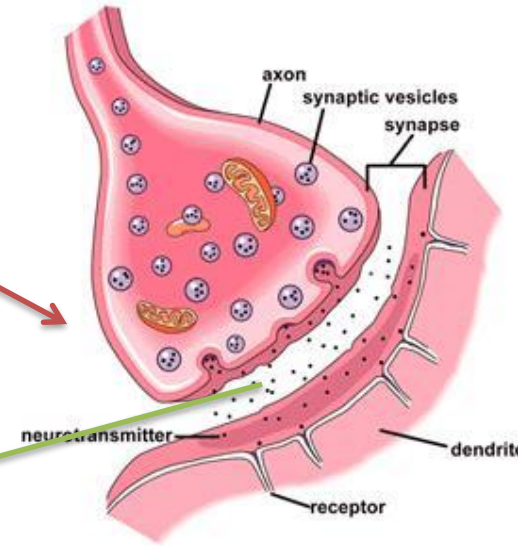
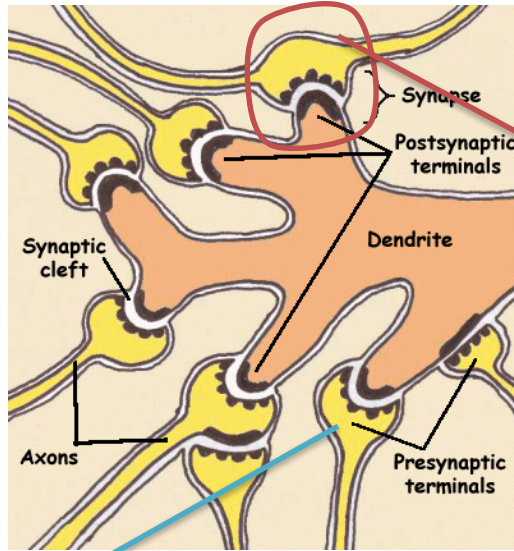


$$o = \Gamma(W_n(\Gamma(\dots W_2\Gamma(W_1x - \theta_1) - \theta_2)\dots) - \theta_n)$$



$$o = F(W, x)$$

- Should arrange the weights/connections so that the network performs a task with acceptable performance...
- Possible tasks : Classification, Pattern Recognition, Approximation,

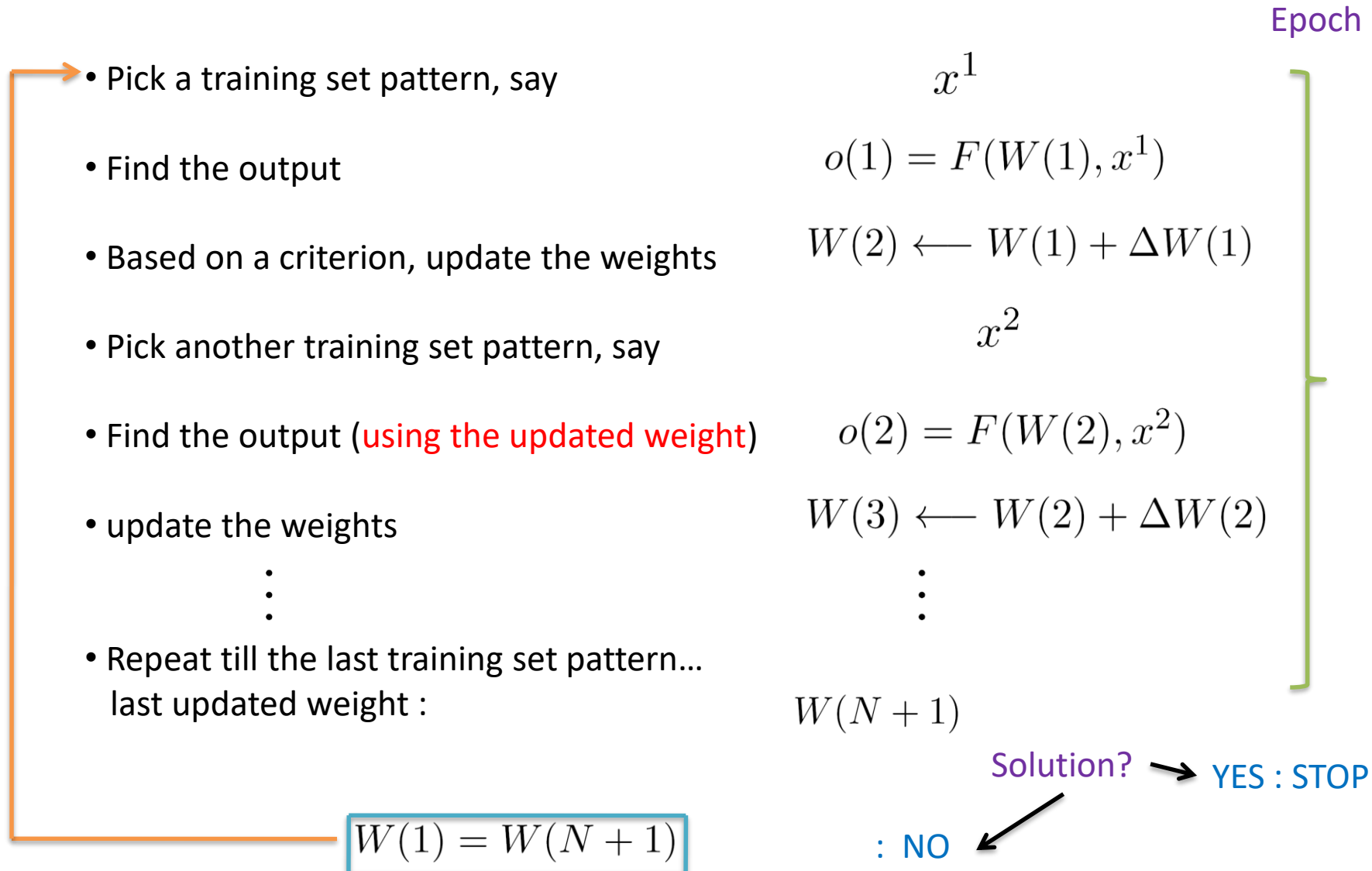


• Problem : Given the task, how to choose connections/weights so that the task is achieved...?

• **LEARNING** : A systematic way of changing the connections/weights to achieve the task...

Given a training set : $\mathcal{S}_{Tr} = \{x^1, x^2, \dots, x^N\}$

- Choose an initial connection/weight $W(1)$



- Selection of weight update rule : ΔW **LEARNING**

- Convergence ?

- Types of Learning?

- **1 : Supervised Learning.** Here, at each pattern in the training set, we know what the correct output should be. And we decide on ΔW accordingly. This is called **learning with a teacher**.

- **2 : Unsupervised Learning.** Here, we do not know the correct output for the training patterns. The network should organize itself → **Self-Organization**

- **3 : Reinforcement Learning.** Here, we have an **idea** about the **good** outputs, and **bad** outputs, and we decide on ΔW accordingly (reward and punishment) → game learning.

- In terms of updating, there are 2 different types :

• **1 : Stepwise Learning** One input is given, weights are updated. Then in the next input, last updated weights are used. In this case, there is no need to store the weight increments. This is sometimes called **on-line learning**.

⋮

- Pick a training set pattern, say

$$x^1$$

- Find the output

$$o(1) = F(W(1), x^1)$$

- Based on a criterion, update the weights

$$W(2) \leftarrow W(1) + \Delta W(1)$$

- Pick another training set pattern, say

$$x^2$$

- Find the output (**using the updated weight**)

$$o(2) = F(W(2), x^2)$$

- update the weights

$$W(3) \leftarrow W(2) + \Delta W(2)$$

⋮

- **2 : Batch Learning** One input is given, necessary weight increments are found, **but the weights are not updated**. When the next input is given, previous weights are used, and again the necessary weight increments are found. This process is repeated till the end of an epoch. At the end of epoch, the weight increments are added to find the total increment, and added to the weights used within the epoch. This is sometimes called as **off-line learning**.

Epoch

- Pick a training set pattern, say
- Find the output
- Based on a criterion, find the update
- Pick another training set pattern, say
- Find the output (**using the initial weight**)
- find the update
- $\vdots$
- Repeat till the last training set pattern...
last weight update :

$$x^1$$

$$o(1) = F(W(1), x^1)$$

$$\Delta W(1)$$

$$x^2$$

$$o(2) = F(W(2), x^2)$$

$$\Delta W(2)$$

$$\vdots$$

$$\Delta W(N)$$

$$\Delta W = \sum_{i=1}^N \Delta W(i)$$
$$W(1) \leftarrow W(1) + \Delta W$$

: NO

Solution? →



Supervised Learning

- Each training case consists of an input vector x and a desired output y (there may be multiple desired outputs but we will ignore that for now)
 - Regression: Desired output is a real number
 - Classification: Desired output is a class label (1 or 0 is the simplest case).
- We start by choosing a model-class
 - A model-class is a way of using some numerical parameters, W , to map each input vector, x , into a predicted output y

Linear neurons

- The neuron has a real-valued output which is a weighted sum of its inputs

$$\hat{y} = \sum_i w_i x_i = \mathbf{w}^T \mathbf{x}$$

↑
Neuron's estimate of the desired output

weight vector
↓

input vector
↑

- The aim of learning is to minimize the discrepancy between the desired output and the actual output
 - How do we measure the discrepancies?
 - Do we update the weights after every training case?

A simple example

- Each day you get lunch at the cafeteria.
 - Your diet consists of fish, chips, and soda.
 - You get several portions of each
- The cashier only tells you the total price of the meal
 - After several days, you should be able to figure out the price of each portion.
- Each meal price gives a linear constraint on the prices of the portions:

$$price = x_{fish} w_{fish} + x_{chips} w_{chips} + x_{soda} w_{soda}$$

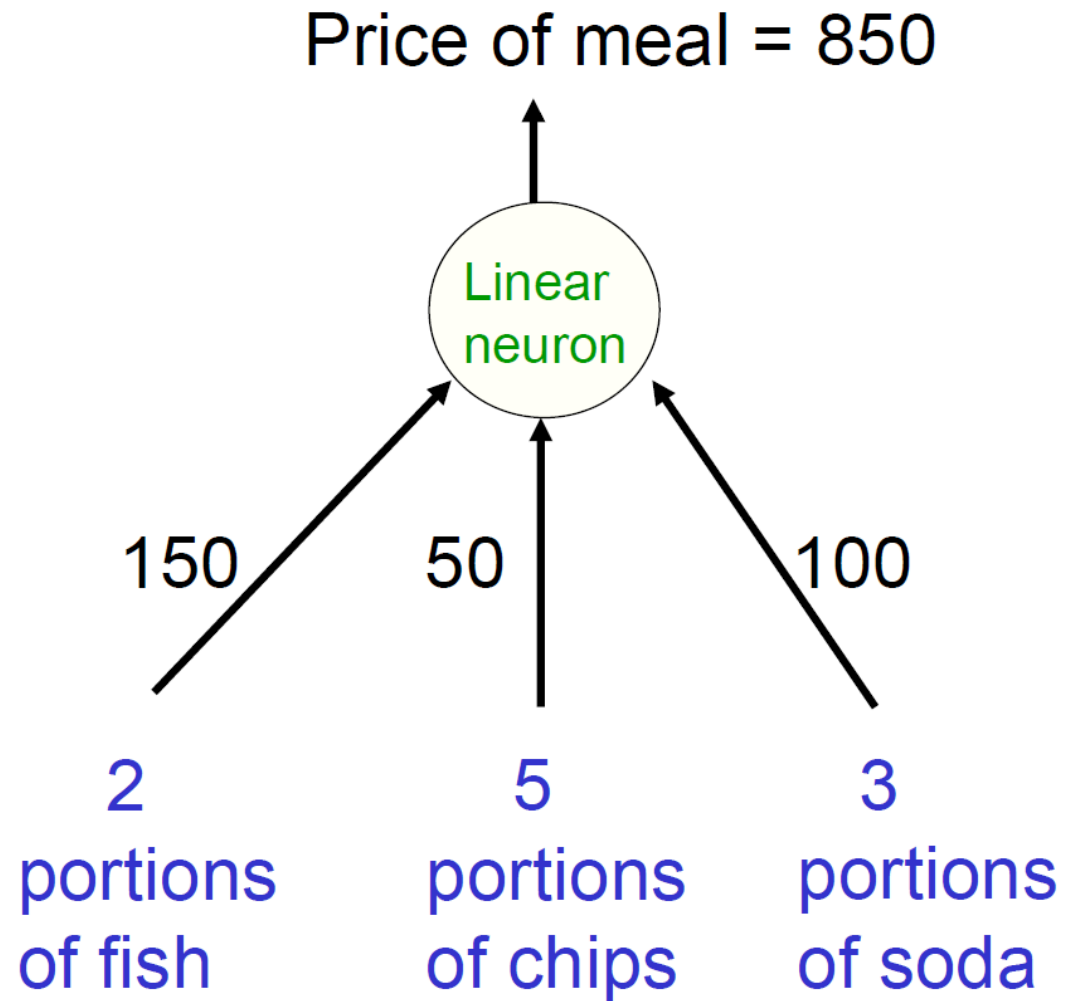
Two ways to solve the equations

- The obvious approach is just to solve a set of simultaneous linear equations, one per meal.
- But we want a method that could be implemented in a neural network.
- The prices of the portions are like the weights in of a linear neuron.

$$\mathbf{w} = (w_{fish}, w_{chips}, w_{soda})$$

- We will start with guesses for the weights and then adjust the guesses to give a better fit to the prices given by the cashier.

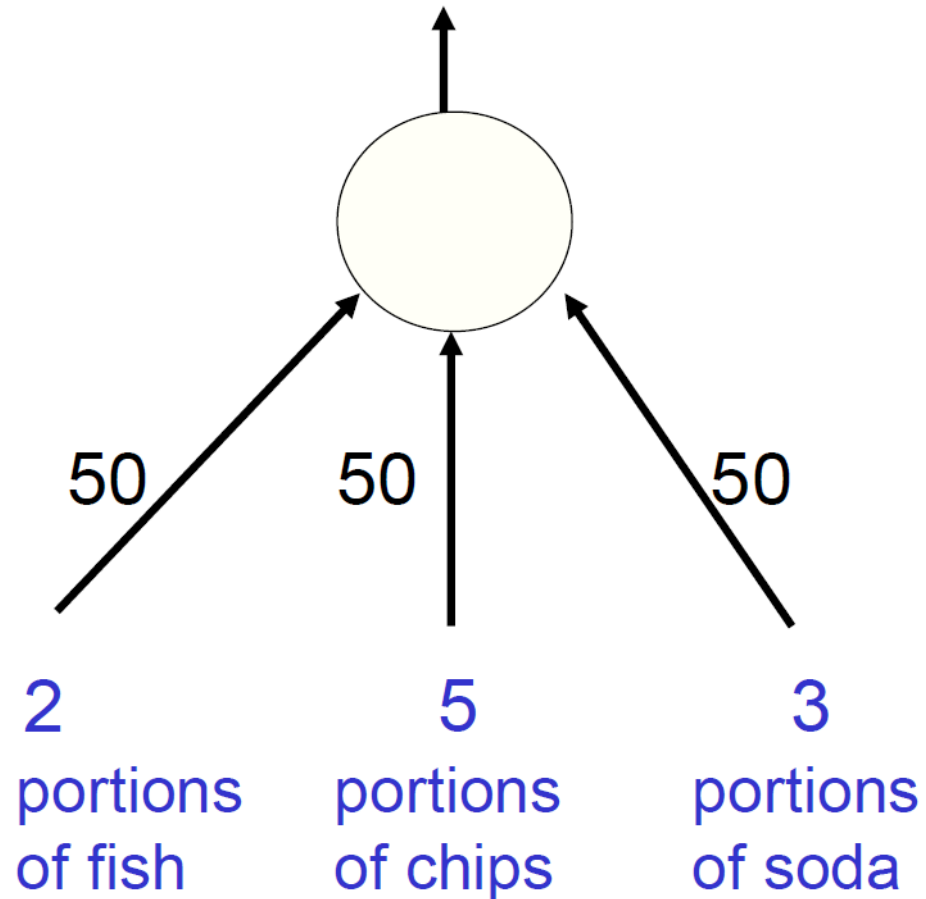
The cashier's brain



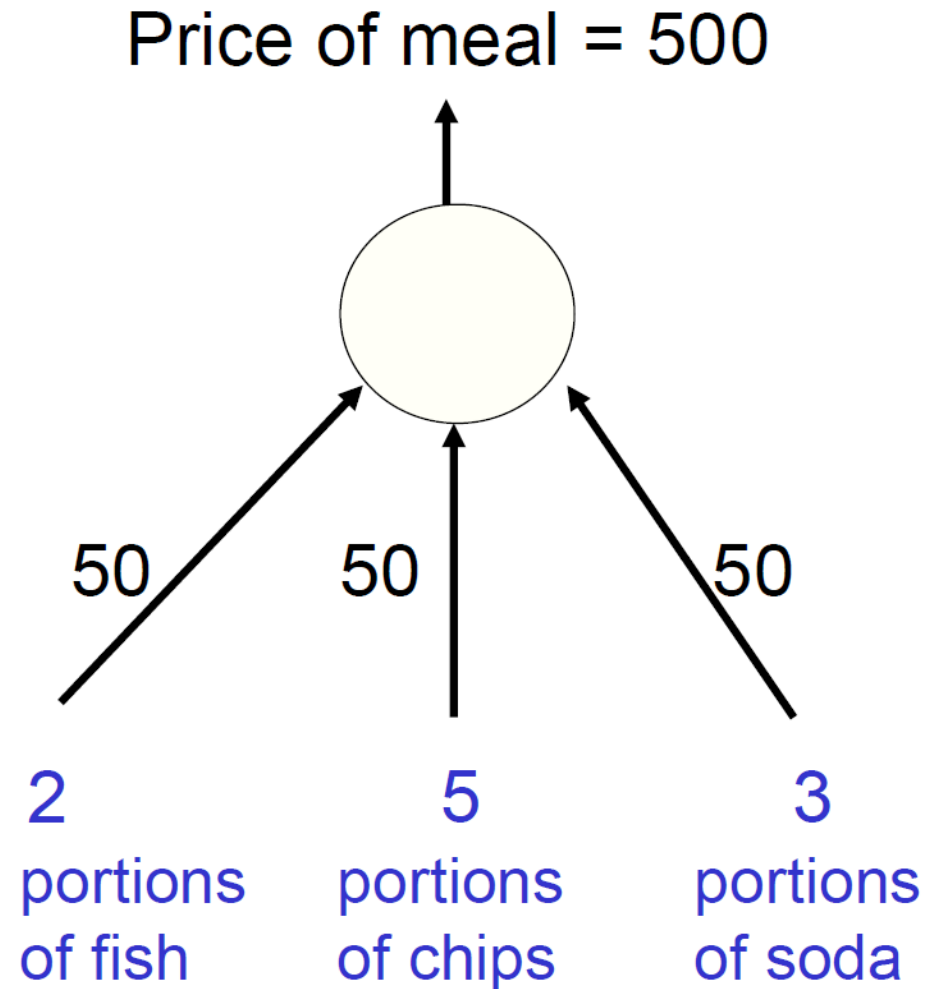
A model of the cashier's brain with arbitrary initial weights

Price of meal = 500

- Residual error = 350



A model of the cashier's brain with arbitrary initial weights



- Residual error = 350
- The learning rule is:
$$\Delta w_i = \varepsilon x_i (y - \hat{y})$$
- With a learning rate ε of 1/35, the weight changes are +20, +50, +30
- This gives new weights of 70, 100, 80

• Supervised Learning

- Approximation problem ...

$$h : \mathbf{R}^n \rightarrow \mathbf{R}^m$$

$$\mathcal{S}_{Tr} = \{x^1, x^2, \dots, x^N\}$$

Training Set

x



$h(x)$

$$\{h(x^1), h(x^2), \dots, h(x^N)\}$$

Desired output (Sampling)



$$o(x) = H(W, x)$$

- The learning problem is : Find a weight W_* such that $H(W_*, x)$ is as close to the $h(x)$ over the training set as possible.

- How to measure closeness?

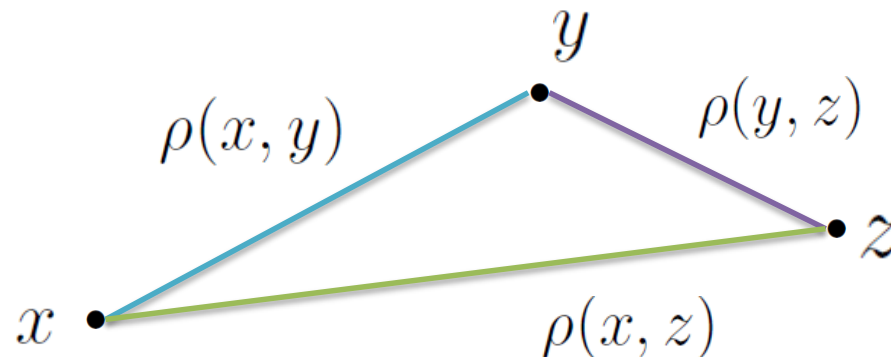


(distance) **METRIC**

- Given vectors $x, y \in \mathbf{R}^n$, a metric ρ is a function if it satisfies

- **1** : $\rho(x, y) = \rho(y, x) \geq 0$, and $\rho(x, y) = 0 \Leftrightarrow x = y$
- **2** : $\rho(\alpha x, \alpha y) = |\alpha| \rho(x, y) \quad \alpha \in \mathbf{R}$
- **3** : $\rho(x, z) \leq \rho(x, y) + \rho(y, z)$ (Triangle inequality).

- We may treat $\rho(x, y)$ as the distance between the vectors x and y .



$$x = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \quad y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

$$\rho(x, y) = \sqrt{(x_1 - y_1)^2 + \dots + (x_n - y_n)^2} = \|x - y\|$$

Euclidean distance L_2 **distance (norm)**

$$\rho(x, y) = |x_1 - y_1| + \dots + |x_n - y_n|$$

L_1 **distance (norm)**

$$\rho(x, y) = \max\{|x_1 - y_1|, \dots, |x_n - y_n|\}$$

L_∞ **distance (norm)**

- We will prefer **Euclidean distance**, since it is differentiable with respect to x and y . Note that $\|x - y\|^2 = (x - y)^T(x - y)$

- So, the learning problem is : given a metric ρ , and a training set $\mathcal{T}_r$, find a weight vector W_* such that

- $\rho(H(W_*, x), h(x)) \leq \rho(H(W, x), h(x))$ for any $x \in \mathcal{T}_r$ and for any other weight vector W .



$$\min_W \rho(o(W, x), h(x)) \quad x \in \mathcal{T}_r$$



Minimization problem !

training set $\mathcal{T}_r = \{x^1, x^2, \dots, x^N\}$

For pattern x^i , the approximation error : $E_i = \|H(W, x^i) - h(x^i)\|^2$

L_2 norm

$$E(W) = \frac{1}{N} \sum_{i=1}^N E_i = \frac{1}{N} \sum_{i=1}^N \|H(W, x^i) - h(x^i)\|^2$$

Average error over training set

Find a weight W_* such that :

$$E(W_*) \leq E(W) \text{ for any } W \neq W_*$$

- How to update the weights?

GRADIENT DESCENT

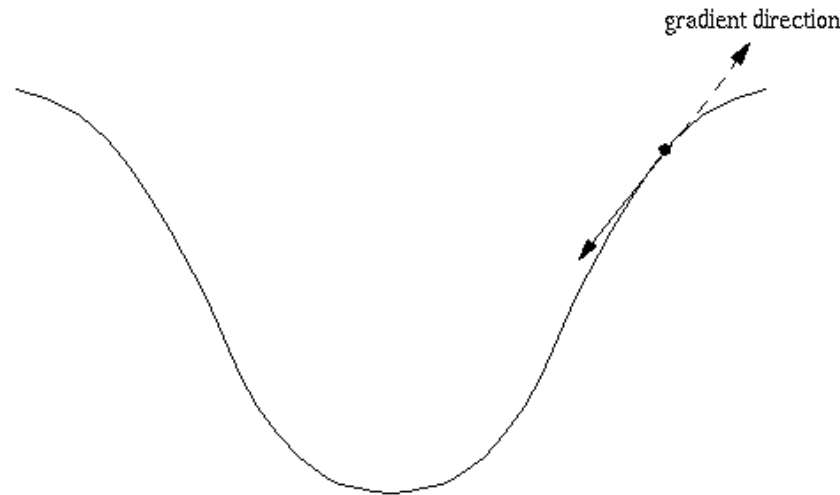
$$E(W + \Delta W) = E(W) + \frac{\partial E}{\partial W} \Delta W + \text{h.o.t.}$$

$$\text{Choose } \Delta W = -\eta \frac{\partial E}{\partial W}$$

$$E(W + \Delta W) = E(W) - \eta \left\| \frac{\partial E}{\partial W} \right\|^2 + \text{h.o.t.}$$

$$E(W + \Delta W) \leq E(W)$$

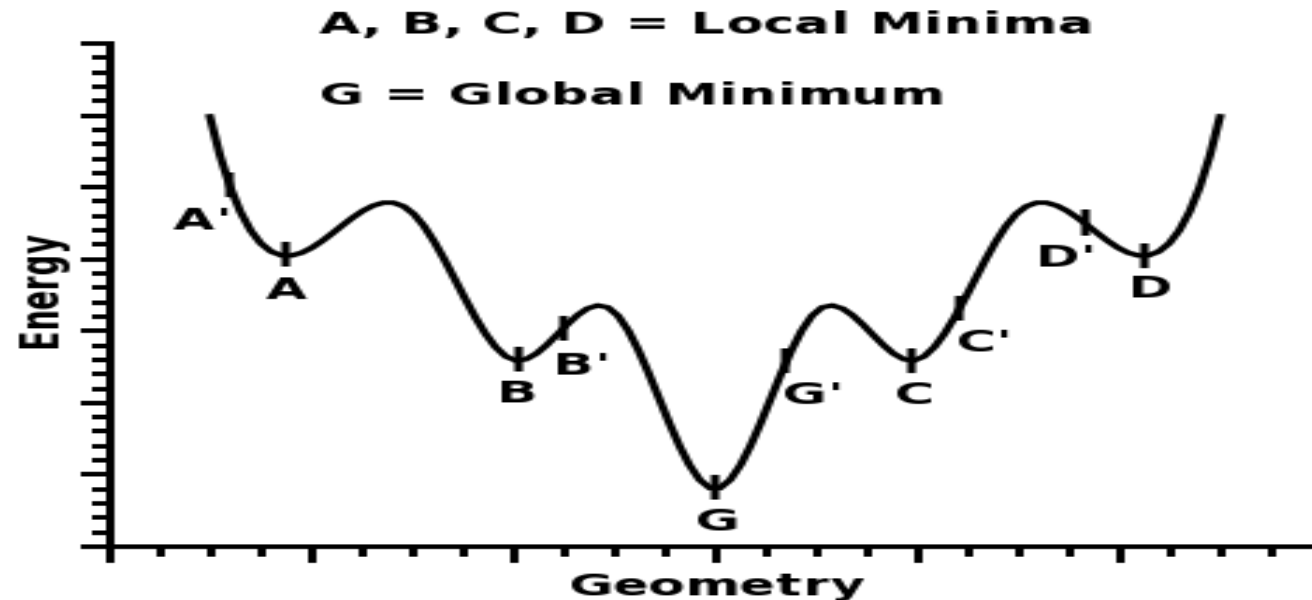
will stop when $\frac{\partial E}{\partial W} = 0 \rightarrow$ **local minimum.**



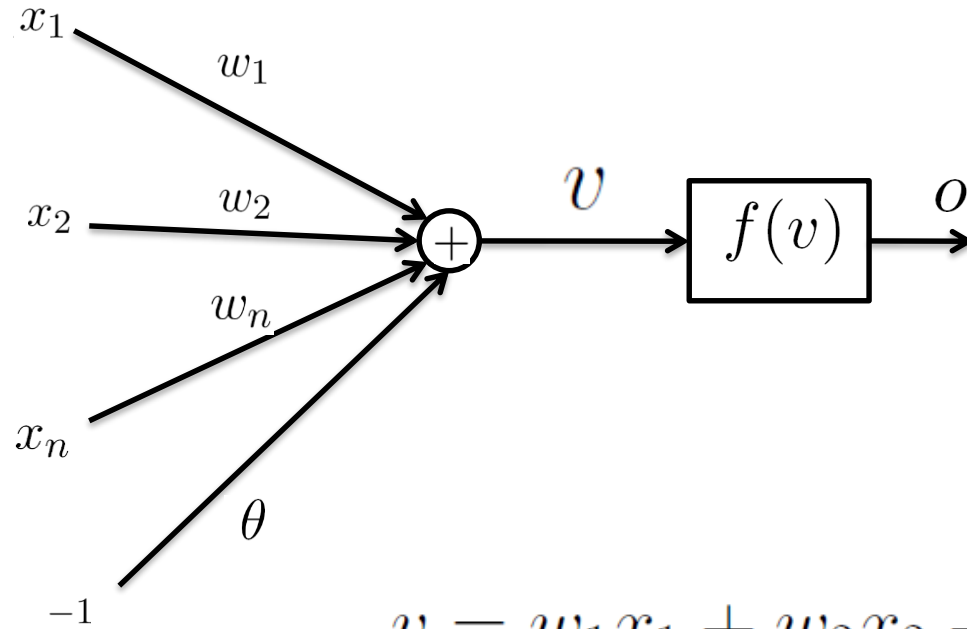
- If $\eta > 0$ is sufficiently small, and if initial weight $W(1)$ is sufficiently close to a local minimum W_* , the above update law will always converge to the local minimum.

- This idea is the basic principle behind many optimization techniques, and also for the back propagation algorithm.

There may be many local minima. Global minimum?



- Supervised learning rules
- **perceptron** learning rule was one of such rules.
- **Delta Learning Rule**



$$v = w_1x_1 + w_2x_2 + \dots + w_nx_n - \theta$$

$$o = f(v)$$

- $\mathbf{x}$ is applied (training set), desired output is $\mathbf{d}$

$$\Delta W_e = c \underbrace{[d - o] f'(v)}_{\text{Learning Signal}} x_e \quad r = [d - o] f'(v)$$

$$\Delta w_i = c[d - o] f'(v) x_i \quad i = 1, 2, \dots, n + 1$$

$$W_{n+1} = \theta, \quad x_{n+1} = -1$$

- **Fact :** This rule tries to minimize $e = 0.5(d - o)^2$.
- **Proof :** Remember the gradient descent

$$\bullet \Delta w_i = -\eta \frac{\partial e}{\partial w_i}$$

- Chain of dependence : $e \rightarrow o \rightarrow v \rightarrow w_i$

$$\frac{\partial e}{\partial w_i} = \frac{\partial e}{\partial o} \frac{\partial o}{\partial v} \frac{\partial v}{\partial w_i}$$

(LEIBNIZ RULE)- Chain Rule

- $\frac{\partial e}{\partial o} = -(d - o)$

- $o = f(v) \longrightarrow \frac{\partial o}{\partial v} = f'(v)$

- $v = w_1x_1 + w_2x_2 + \dots + w_nx_n - \theta \longrightarrow \frac{\partial v}{\partial w_i} = x_i$

$$\bullet \Delta w_i = -\eta \frac{\partial e}{\partial w_i} = -\eta \frac{\partial e}{\partial o} \frac{\partial o}{\partial v} \frac{\partial v}{\partial w_i} = \eta [d - o] f'(v) x_i$$

- Computation of $f'(v)$

- Bipolar sigmoidal :

- $f(v) = \frac{1-e^{-\lambda v}}{1+e^{-\lambda v}} \quad f'(v) = \frac{\lambda e^{-\lambda v}(1+e^{-\lambda v}) + \lambda e^{-\lambda v}(1-e^{-\lambda v})}{(1+e^{-\lambda v})^2} = \frac{2\lambda e^{-\lambda v}}{(1+e^{-\lambda v})^2}$
- $1 - f^2(v) = 1 - \frac{(1-e^{-\lambda v})^2}{(1+e^{-\lambda v})^2} = \frac{(1+e^{-\lambda v})^2 - (1-e^{-\lambda v})^2}{(1+e^{-\lambda v})^2} = \frac{4e^{-\lambda v}}{(1+e^{-\lambda v})^2}$
- $f'(v) = \frac{\lambda}{2} (1 - f^2(v)) = \frac{\lambda}{2} (1 - o^2)$

- Unipolar sigmoidal :

- $f(v) = \frac{1}{1+e^{-\lambda v}}$
- $f'(v) = \frac{\lambda e^{-\lambda v}}{(1+e^{-\lambda v})^2} = \lambda \frac{1}{1+e^{-\lambda v}} \frac{e^{-\lambda v}}{1+e^{-\lambda v}}$
- $1 - f(v) = (1 - o) = \frac{e^{-\lambda v}}{1+e^{-\lambda v}}$
- $f'(v) = \lambda o(1 - o)$

- Usually we have a training set $\mathcal{S}_{Tr} = \{x^1, x^2, \dots, x^N\}$ and we have a set of desired outputs $d^1, \dots, d^N$.

the error made at i th pattern e_i $e_i = 0.5(d^i - o(i))^2$

$$\begin{aligned} \bullet \quad E_{cum} &= \sum_{i=1}^N e_i & E_{ave} &= \frac{1}{N} E_{cum} \\ \bullet \quad \Delta w &= -\eta \frac{\partial E_{cum}}{\partial w} = -\eta \sum_{i=1}^N \frac{\partial e_i}{\partial w} \end{aligned}$$

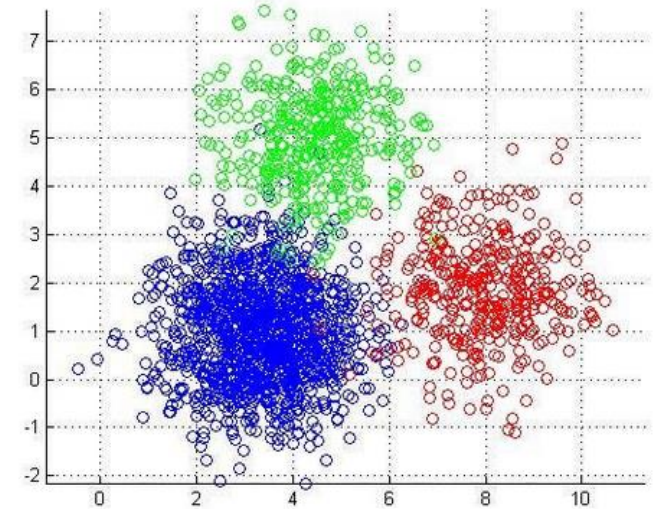
- This could be achieved if we use **batch mode** of learning
- For practical reasons, we do not use the batch mode, and use **incremental learning** rule as a modification :

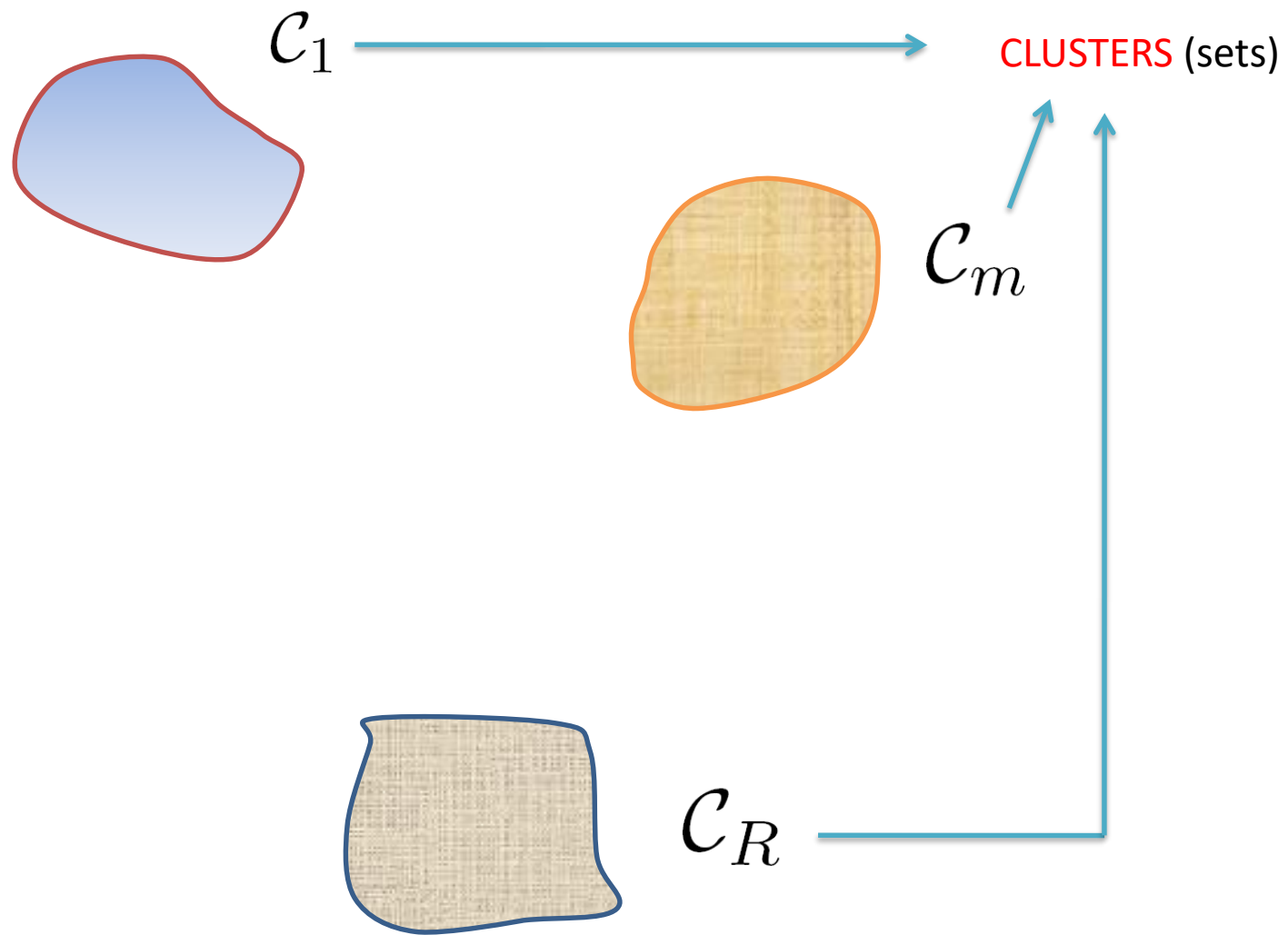
● Unsupervised Learning

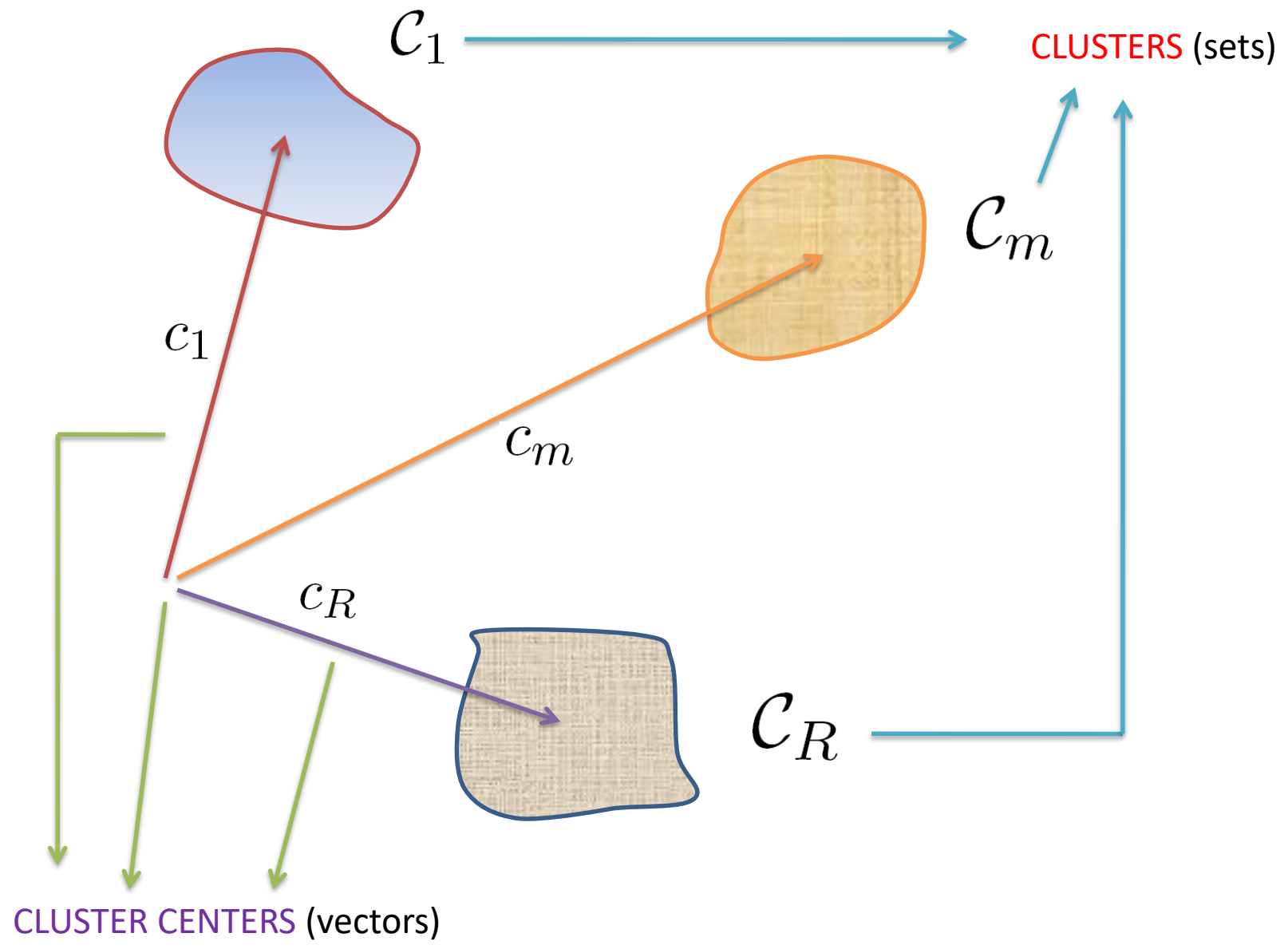
- SELF ORGANIZATION
- Training Set $\mathcal{T}_r$ is given ...
- Desired (correct) output ?
- NN should **ORGANIZE** itself (How ?)
- Detecting similarities between patterns ?
- Clustering ?
- Typical application : classification

● Unsupervised Learning

- SELF ORGANIZATION
- Training Set $\mathcal{T}_r$ is given ...
- Desired (correct) output ?
- NN should **ORGANIZE** itself (How ?)
- Detecting similarities between patterns ?
- Clustering ? 
- Typical application : classification



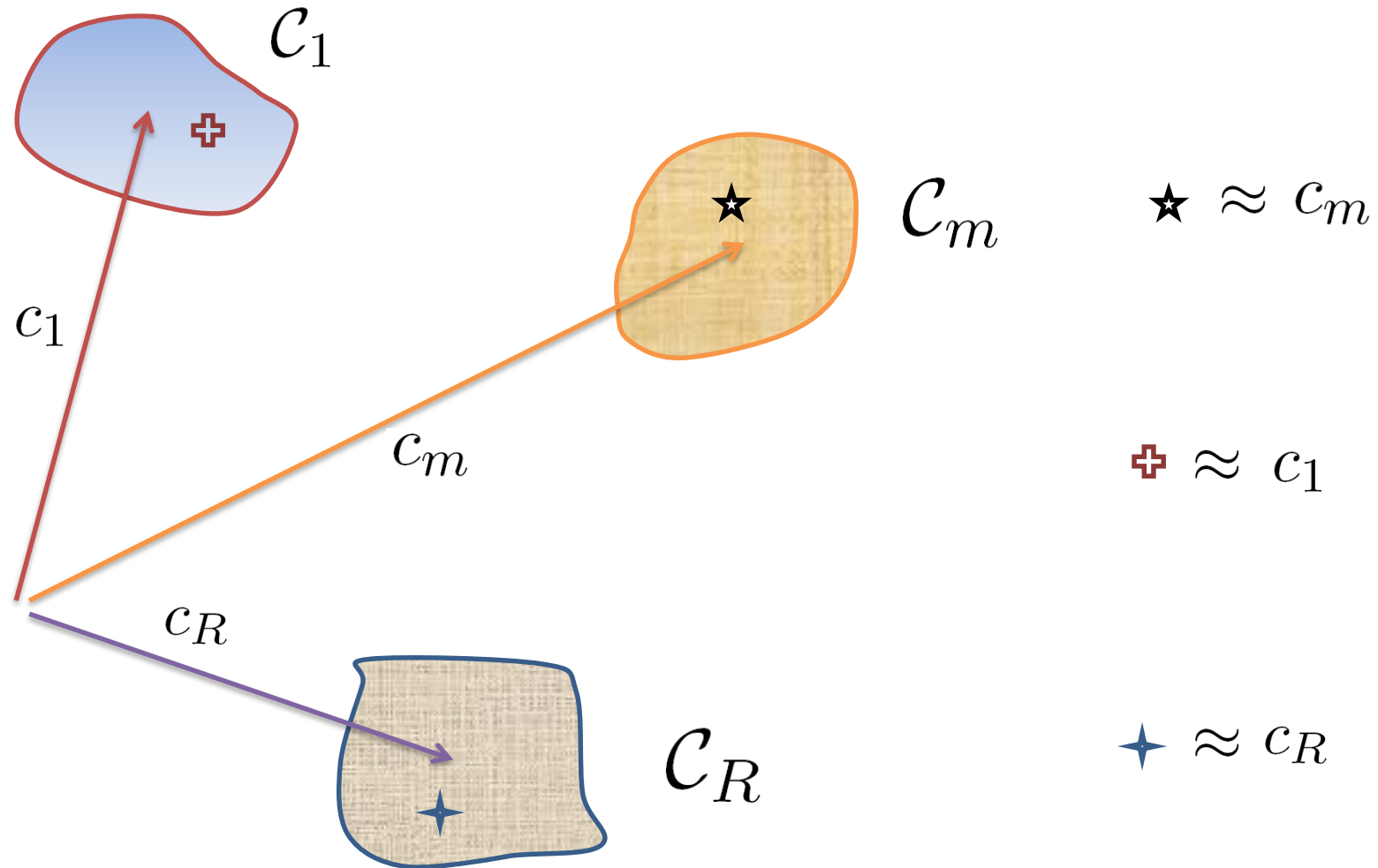




$\mathcal{C}_i$ will have a **cluster center** c^i

Idea : Measure the DISTANCE to CLUSTER CENTERS !

$$\|x^s - c^i\| < \|x^s - c^j\| \text{ for any } j \quad \Leftrightarrow \quad x^s \in \mathcal{C}_i$$



PROBLEM : How to find appropriate CLUSTER CENTERS?

Here, c^i are also called **quantization vectors**.

Unsupervised learning tries to minimize **quantization error**

Assume R clusters and N training set patterns

$$E = \sum_{i=1}^N \left\| x^i - \underbrace{\operatorname{argmin}_{c^j} \|x^i - c^j\|}_{\text{QUANTIZATION ERROR for } x^i} \right\|$$

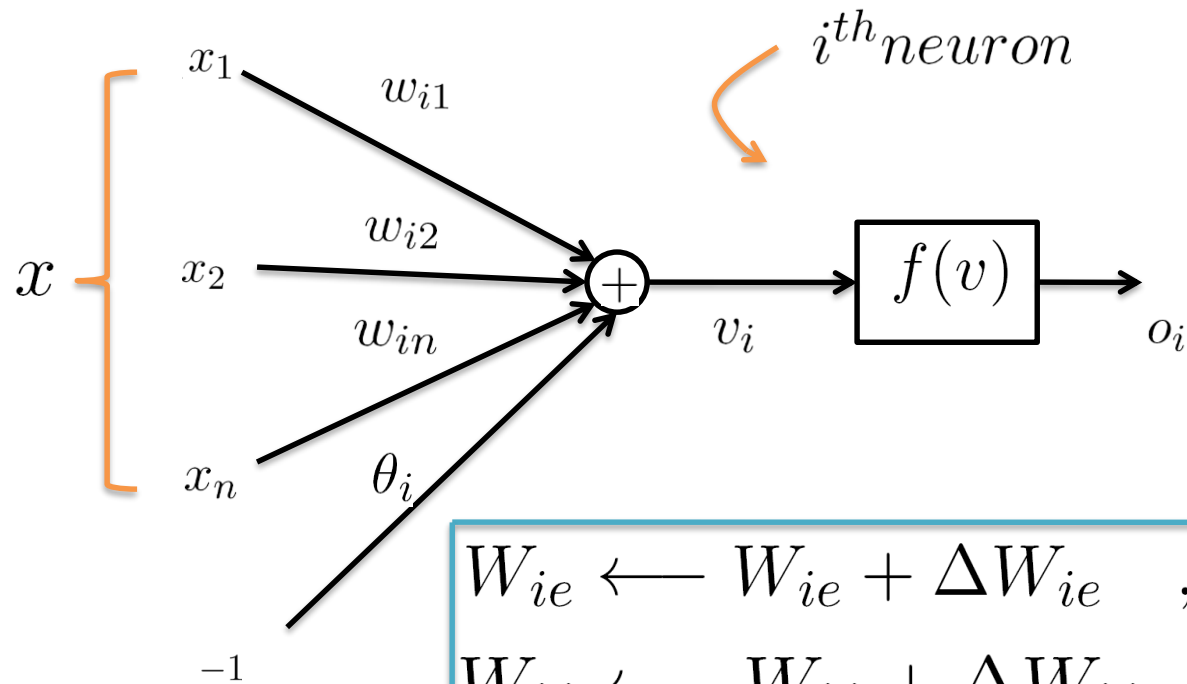
CLUSTER CENTER

Total Quantization Error

$$\min_{\{c^j\}} E$$

not differentiable !

General Form of Learning rules



$$W_{ie} \leftarrow W_{ie} + \Delta W_{ie} \quad , \quad \Delta W_{ie} = crx_e$$

$$W_{ij} \leftarrow W_{ij} + \Delta W_{ij} \quad , \quad \Delta W_{ij} = crx_j$$

$$j = 1, 2, \dots, n, n+1 \quad W_{i,n+1} = \theta_i \quad x_{n+1} = -1$$

$c > 0$ is a learning constant.

$r = r(W_i(n), d, x)$ is a **scalar** learning signal.

d : DESIRED OUTPUT (exists only in SUPERVISED case)

• Hebbian Learning Rule

proposed by D. Webb in 1940's.

• *When an axon of a cell A is near enough to excite a cell B and repeatedly or persistently fires it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased.*

$$\Delta W_{ie} = co_i x_e \quad \Rightarrow \quad \Delta W_{ij} = co_i x_j$$

$o_i x_j > 0 \Rightarrow W_{ij} \uparrow$

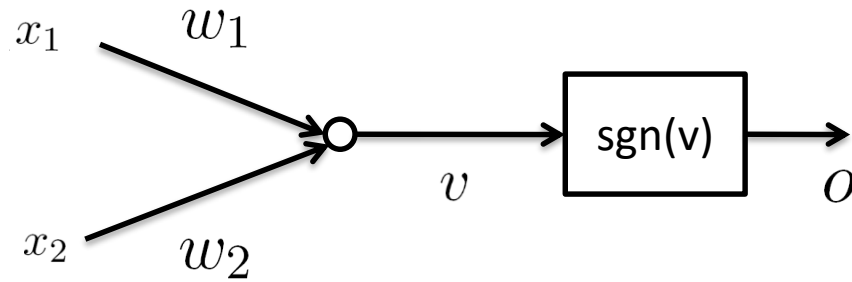
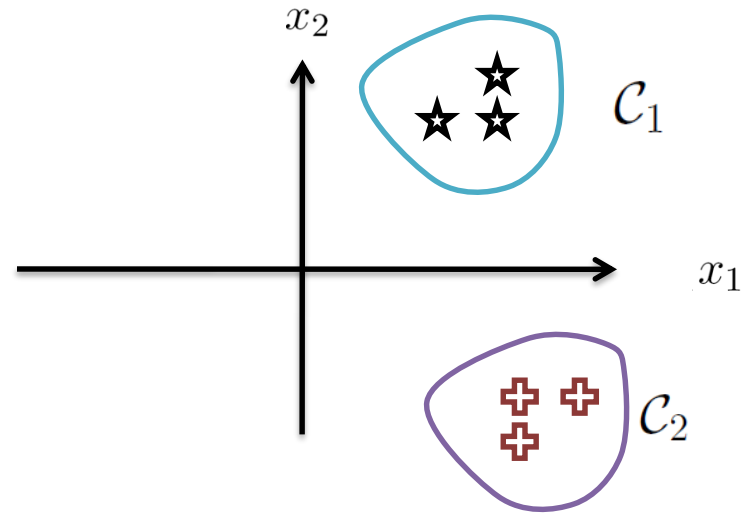
$o_i x_j < 0 \Rightarrow W_{ij} \downarrow$

<https://www.apa.org/monitor/julaug02/eminent>

saturation problem

$$\Delta W_{ij}(n) = co_i(n)x_j - \alpha W_{ij}(n) \quad \alpha > 0$$

$$\mathcal{C}_1 = \left\{ x^1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, x^2 = \begin{pmatrix} 0.9 \\ 1 \end{pmatrix}, x^3 = \begin{pmatrix} 1 \\ 1.1 \end{pmatrix} \right\} \quad \mathcal{C}_2 = \left\{ x^4 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, x^5 = \begin{pmatrix} 1.1 \\ -1 \end{pmatrix}, x^6 = \begin{pmatrix} 1 \\ -1.1 \end{pmatrix} \right\}$$



choose $c = 1$

$$w_1(1) = w_2(1) = 0.1$$

• **begin epoch 1 :**

$$\bullet x^1 \rightarrow o(1) = \text{sgn}(0.1 + 0.1) = 1 \rightarrow W(2) = W(1) + 1 \times 1 \times x^1 = \begin{pmatrix} 1.1 \\ 1.1 \end{pmatrix}$$

$$\bullet x^2 \rightarrow o(2) = \text{sgn}(W(2)^T x^2) = 1 \rightarrow W(3) = W(2) + 1 \times 1 \times x^2 = \begin{pmatrix} 2 \\ 2.1 \end{pmatrix}$$

$$\bullet x^3 \rightarrow o(3) = \text{sgn}(W(3)^T x^3) = 1 \rightarrow W(4) = W(3) + 1 \times 1 \times x^3 = \begin{pmatrix} 3 \\ 3.2 \end{pmatrix}$$

$$\bullet x^4 \rightarrow o(4) = \text{sgn}(W(4)^T x^4) = -1 \rightarrow W(5) = W(4) - 1 \times 1 \times x^4 = \begin{pmatrix} 2 \\ 4.2 \end{pmatrix}$$

$$\bullet x^5 \rightarrow o(5) = \text{sgn}(W(5)^T x^5) = -1 \rightarrow W(6) = W(5) - x^5 = \begin{pmatrix} 0.9 \\ 5.2 \end{pmatrix}$$

$$\bullet x^6 \rightarrow o(6) = \text{sgn}(W(6)^T x^6) = -1 \rightarrow W(7) = W(6) - x^6 = \begin{pmatrix} -0.1 \\ 6.3 \end{pmatrix}$$

• **end of epoch 1**

Let us stop and call $W_* = W(7)$.

- $x^1 \rightarrow o = \text{sgn}(W_*^T x^1) = 1$

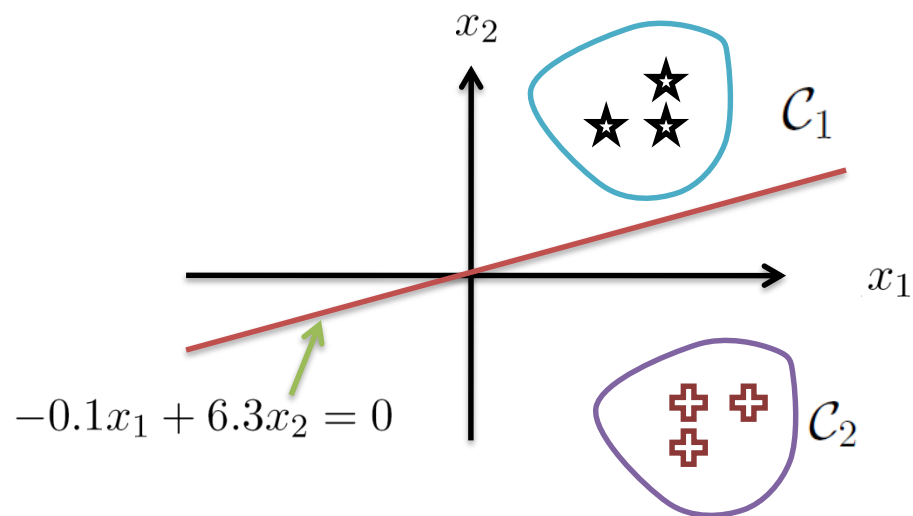
- $x^2 \rightarrow o = \text{sgn}(W_*^T x^2) = 1$

- $x^3 \rightarrow o = \text{sgn}(W_*^T x^3) = 1$

- $x^4 \rightarrow o = \text{sgn}(W_*^T x^4) = -1$

- $x^5 \rightarrow o = \text{sgn}(W_*^T x^5) = -1$

- $x^6 \rightarrow o = \text{sgn}(W_*^T x^6) = -1$

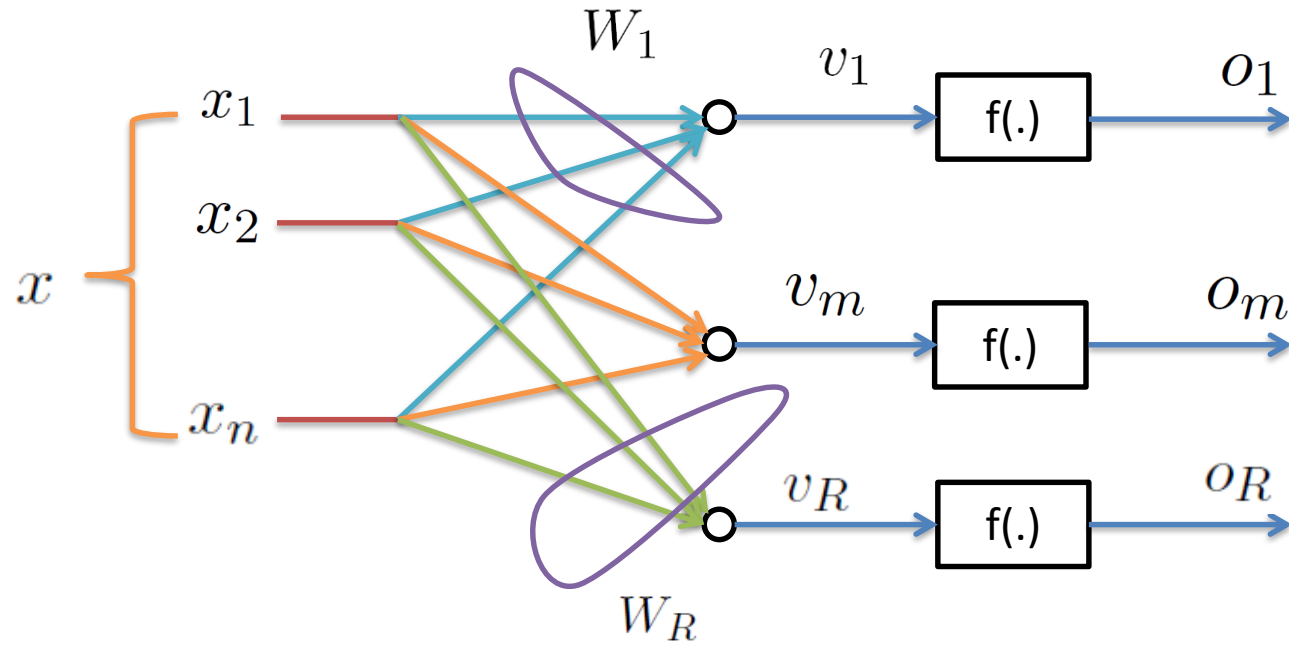


$$o = \text{sgn}(W_*^T x) = \text{sgn}(-0.1x_1 + 6.3x_2)$$

$$o = \begin{cases} 1 & x \in \mathcal{C}_1 \\ -1 & x \in \mathcal{C}_2 \end{cases}$$

- Winner-Take-All Learning Rule

training set $\mathcal{T}_r = \{x^1, x^2, \dots, x^N\}$



Could use the extended version as well...

- Given the pattern x from the training set :
- Compute the LINEAR SUMS :

$$v_i = W_i^T x \quad i = 1, 2, \dots, R$$

- Find the WINNING NEURON :

$$\max\{v_1, v_2, \dots, v_R\}$$

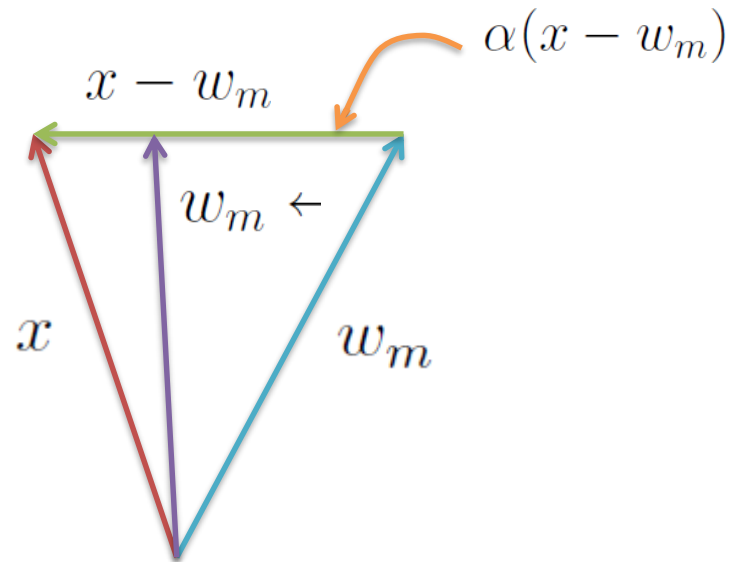
- Assume that m^{th} neuron is WINNER :
- UPDATE ONLY THE WINNING NEURON WEIGHT

$$w_m \longleftarrow w_m + \Delta w_m, \quad \Delta w_m = \alpha(x - w_m)$$

$$w_{mj} \longleftarrow w_{mj} + \Delta w_{mj}, \quad \Delta w_{mj} = \alpha(x_j - w_{mj})$$

- $\alpha > 0$ is learning constant.

EFFECT : Winning neuron weight $w_m \longrightarrow x$



- $0 < \alpha < 1$
- If same x is applied after update, SAME NEURON WINS !

Keep applying the same input !

$$w_m(2) = w_m(1) + \alpha(x - w_m(1)) = (1 - \alpha)w_m(1) + \alpha x$$

$$\begin{aligned} w_m(3) &= w_m(2) + \alpha(x - w_m(2)) \\ &= (1 - \alpha)^2 w_m(1) + \alpha[1 + (1 - \alpha)]x \end{aligned}$$

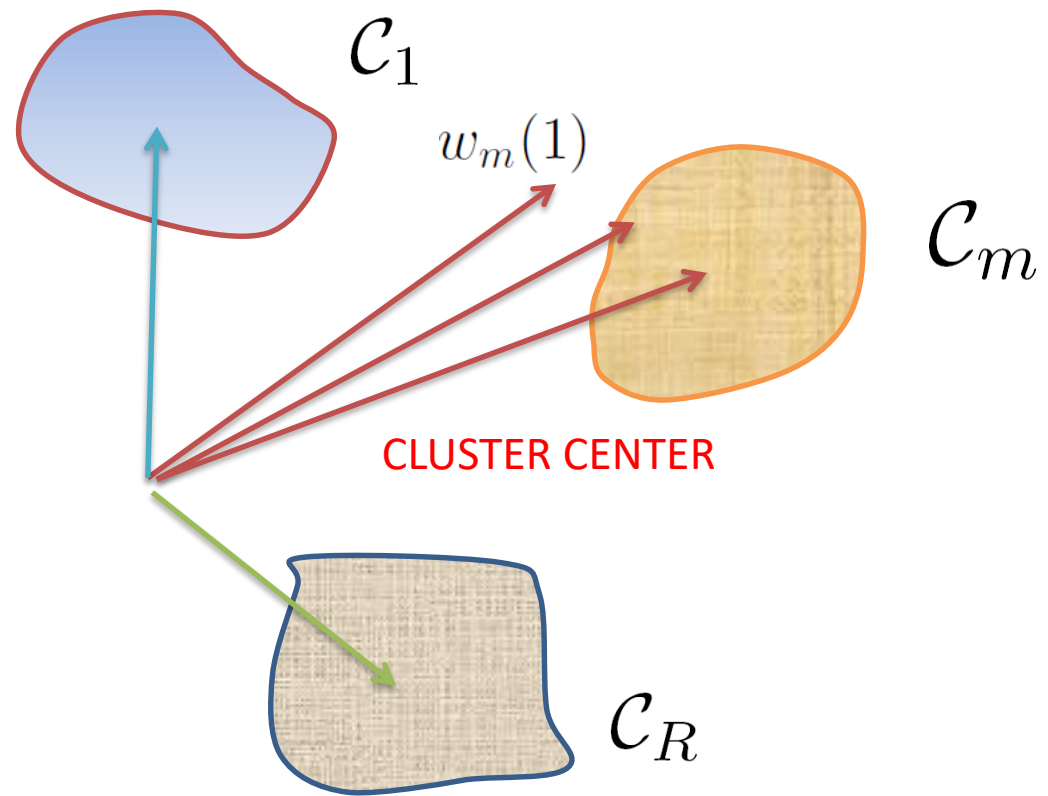
$\vdots$

$$\begin{aligned} w_m(n + 1) &= w_m(n) + \alpha(x - w_m(n)) \\ &= (1 - \alpha)^n w_m(1) + \alpha[1 + (1 - \alpha) + \dots + (1 - \alpha)^{n-1}]x \end{aligned}$$


 $\rightarrow 0$

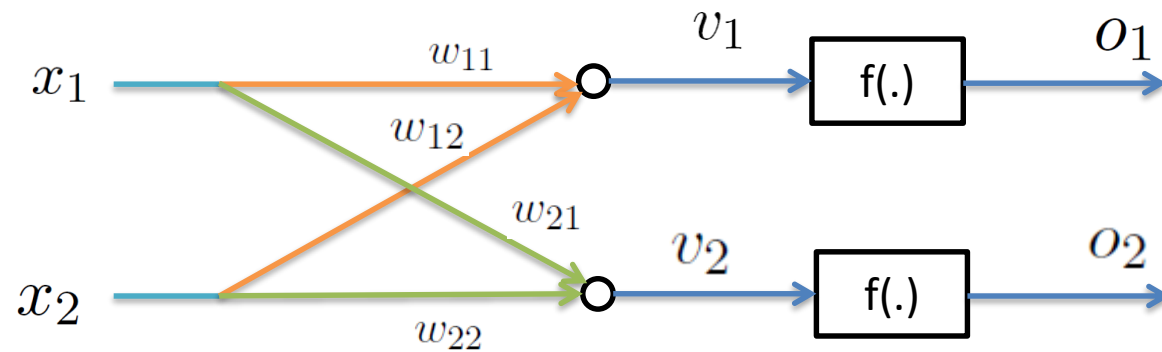
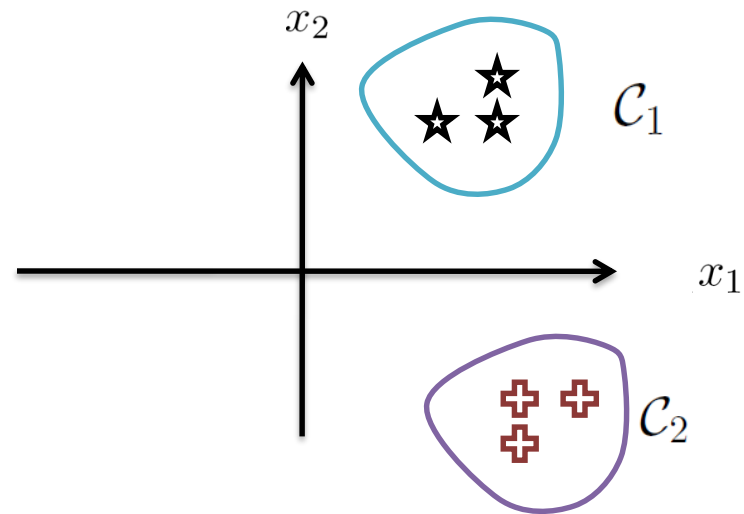
$$1 + (1 - \alpha) + \dots + (1 - \alpha)^{n-1} = \frac{1 - (1 - \alpha)^n}{1 - (1 - \alpha)} \rightarrow 0$$

$$w_m \rightarrow x$$



EXPECTATION : Each winning neuron weight converges to a CLUSTER CENTER

$$\mathcal{C}_1 = \left\{ x^1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, x^2 = \begin{pmatrix} 0.9 \\ 1 \end{pmatrix}, x^3 = \begin{pmatrix} 1 \\ 1.1 \end{pmatrix} \right\} \quad \mathcal{C}_2 = \left\{ x^4 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, x^5 = \begin{pmatrix} 1.1 \\ -1 \end{pmatrix}, x^6 = \begin{pmatrix} 1 \\ -1.1 \end{pmatrix} \right\}$$



choose $\alpha = 0.5$ and $W_1(1) = \begin{pmatrix} 0.1 \\ 0.1 \end{pmatrix}$ $W_2(1) = \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix}$

• **begin epoch 1 :**

• $x^1 \rightarrow v_1(1) = (0.1 + 0.1) = 0.2, \quad v_2(1) = 0.1 - 0.1 = 0$

• $\longrightarrow$ First neuron wins...

• $W_1(2) = W_1(1) + 0.5(x^1 - W_1(1)) = \begin{pmatrix} 0.55 \\ 0.55 \end{pmatrix}, \quad W_2(2) = W_2(1)$

• $x^2 \rightarrow v_1(2) = (W_1(2)^T x^2) = 1.045, \quad v_2(2) = (W_2(2)^T x^2) = -0.01$

• $\longrightarrow$ First neuron wins...

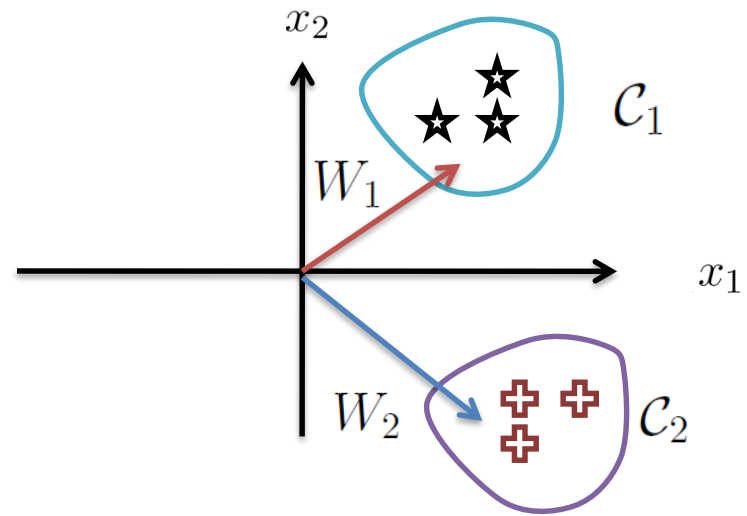
• $\rightarrow W_1(3) = W_1(2) + 0.5(x^2 - W_1(2)) = \begin{pmatrix} 0.725 \\ 0.775 \end{pmatrix}, \quad W_2(3) = W_2(2)$

• $x^3 \rightarrow v_1(3) = (W_1(3)^T x^3) = 1.5775, \quad v_2(3) = (W_2(3)^T x^3) = -0.01$

• $\longrightarrow$ First neuron wins...

• $\rightarrow W_1(4) = W_1(3) + 0.5(x^3 - W_1(3)) = \begin{pmatrix} 0.8625 \\ 0.9375 \end{pmatrix}, \quad W_2(4) = W_2(3)$

- $x^4 \rightarrow v_1(4) = (W_1(4)^T x^4) = -0.075, \quad v_2(4) = W_2(4)^T x^4 = 0.2$
- $\longrightarrow$ Second neuron wins...
- $\rightarrow W_2(5) = W_2(4) + 0.5(x^4 - W_2(4)) = \begin{pmatrix} 0.55 \\ -0.55 \end{pmatrix}, \quad W_1(5) = W_1(4)$
- $x^5 \rightarrow v_1(5) = (W_1(5)^T x^5) = 0.011, \quad v_2(5) = W_2(5)^T x^5 = 1.155$
- $\longrightarrow$ Second neuron wins...
- $\rightarrow W_2(6) = W_2(5) + 0.5(x^5 - W_2(5)) = \begin{pmatrix} 0.825 \\ -0.775 \end{pmatrix}, \quad W_1(6) = W_1(5)$
- $x^6 \rightarrow v_1(6) = (W_1(6)^T x^6) = -0.168, \quad v_2(6) = (W_2(6)^T x^6) = 1.6775$
- $\longrightarrow$ Second neuron wins...
- $\rightarrow W_2(7) = W_2(6) + 0.5(x^6 - W_2(6)) = \begin{pmatrix} 0.9125 \\ -0.9375 \end{pmatrix}, \quad W_1(7) = W_1(6)$
- **end of epoch 1**



For convergence, α should be decreased....